

Flujos de Mensajes — Smpp Gateway

Diagramas de flujo del sistema SMPP Gateway. Todos los diagramas usan Mermaid (renders automático en GitHub/GitLab/VS Code con extensión Mermaid).

1. Envío de SMS vía HTTP (flujo completo)

```
sequenceDiagram
    participant Client as Cliente (HTTP)
    participant API as API Server<br/>:8080
    participant Pipeline as Pipeline
    participant Router as Router
    participant Billing as Billing
    participant Queue as Cola Redis<br/>stream:con:{id}:{prio}
    participant Worker as Worker
    participant Session as Sesión SMPP<br/>outbound
    participant SMSC as SMSC

    Client->>API: POST /api/v1/messages<br/>{"to":"569123","text":"Hola"}
    API->>API: Validar tenant (X-Tenant-ID)
    API->>API: Verificar API key (modo prepaid)
    API->>Pipeline: Enviar(Outgoing)
    Pipeline->>Pipeline: Particionar texto (160 chars/segmento)
    Pipeline->>Router: Route(tenant, msisdn, routingTag)
    Router->>Router: Evaluar reglas por prioridad
    Router-->>Pipeline: RouteResult{connector_id, group}
    Pipeline->>Billing: Reserve(tenant, segments)
    Billing-->>Pipeline: amount (débito pendiente)
    Pipeline->>Pipeline: Crear Message (state=buffered)
    Pipeline->>Queue: Enqueue(Item{connectorID, priority})
    Queue-->>Pipeline: OK
    Pipeline-->>API: HTTP 201 {id, segments}
    API-->>Client: 201 Created

    Queue->>Worker: Consume(Item)
    Worker->>Worker: SetConnector(connectorID)
    Worker->>Session: Send(pdu)
    Session->>SMPP: submit_sm PDU
    SMPP->>SMSC: TCP/IP
    SMSC-->>SMPP: submit_sm_resp (msgId)
    SMPP-->>Worker: Respuesta
    Worker->>Worker: Almacenar smscMsgid
    Worker->>Worker: UpdateState(accepted)
    Worker->>Billing: Debit(tenant, amount)
    Worker->>Worker: Ack(Item)
```

2. Recepción de DLR (Delivery Report)

```
sequenceDiagram
    participant SMSC as SMSC
    participant Session as Sesión SMPP<br/>outbound
    participant Worker as Worker
    participant DLRProcessor as dlr.Processor
    participant DLRCache as dlr.Cache<br/>(Redis + Memoria)
```

```

participant Store as Store<br/>(PostgreSQL)
participant ESME as ESME Server<br/>(entrante)
participant ClientESME as Cliente ESME<br/>(externo)

SMSC->>Session: deliver_sm (DLR)
Session->>Worker: DLR event (msgId, stat)
Worker->>DLRProcessor: Process(msgId, stat)
DLRProcessor->>DLRCache: Lookup(msgId)

alt DLR en memoria/Redis
    DLRCache-->>DLRProcessor: found (tenantID, msisdn)
else DLR no encontrado
    DLRProcessor->>Store: GetMessage(msgId)
    Store-->>DLRProcessor: Message
end

DLRProcessor->>Store: UpdateState(msgId, finalState)
DLRProcessor->>DLRProcessor: MapStat(stat) → state
Note over DLRProcessor: ENROUTE→ACCEPTED<br/>DELIVERED→DELIVRD<br/>EXPIRED→EXPIRED<br/>REJECTED→REJECTD

opt Notificación webhook
    DLRProcessor->>DLRProcessor: POST webhook del tenant
end

opt DLR cross-process (ESME entrante)
    DLRProcessor->>ESME: Notify(event)
    ESME->>ClientESME: deliver_sm (DLR)
end

```

3. Reconciliación de DLR

```

sequenceDiagram
    participant Reconciler as dlr.Reconciler<br/>(cron cada 60s)
    participant Store as Store<br/>(PostgreSQL)

    loop Cada 60 segundos
        Reconciler->>Store: ListStaleAccepted(maxAge=5min)
        Store-->>Reconciler: Messages (state=accepted sin DLR)

        loop Por cada mensaje
            Reconciler->>Store: UpdateState(msgId, "expired")
            Reconciler->>Reconciler: Log: msgId expirado sin DLR
        end
    end
end

```

4. Conexión con Proveedor SMSC (conector outbound)

```

sequenceDiagram
    participant Config as Config<br/>(SMG_CONNECTOR_ID)
    participant Worker as Worker
    participant Session as Sesión SMPP
    participant SMSC as SMSC<br/>(Proveedor)

    Config->>Worker: NewWorker(queue, repo)
    Worker->>Session: New(config{host, port, systemID, password})

```

```

loop Conexión
  Session->>SMPP: TCP Connect
  SMPP->>SMSC: bind_transceiver (systemID, password)
  SMSC-->>SMPP: bind_transceiver_resp (OK/ERROR)

  alt Bind exitoso
    Session->>Session: Estado: bound
    loop Reader loop
      SMSC-->>SMPP: PDU (deliver_sm, enquire_link)
      SMPP->>Session: Process PDU
    end
  else Bind fallido
    Session->>Session: Retry after backoff
  end
end

loop Reconexión automática
  Session->>Session: On disconnect
  Session->>Session: Wait backoff (exponential)
  Session->>SMPP: Reconnect
end

```

5. Conexión ESME Entrante (proveedor cliente se conecta a nosotros)

```

sequenceDiagram
  participant ExternalESME as ESME Externo<br/>(Proveedor cliente)
  participant Gateway as SMPP Gateway<br/>ESME Server :2775
  participant Auth as Auth Store
  participant Pipeline as Pipeline
  participant Router as Router
  participant Queue as Cola
  participant Worker as Worker
  participant SMSC as SMSC final

  ExternalESME->>Gateway: TCP Connect
  ExternalESME->>Gateway: bind_transceiver (systemID, password)
  Gateway->>Auth: Lookup Tenant(smpp_systemID)
  Auth-->>Gateway: Tenant (status, smpp_password)

  alt Credenciales válidas
    Gateway-->>ExternalESME: bind_transceiver_resp (ESME_R0K)
    loop Sesión activa
      ExternalESME->>Gateway: submit_sm (to, text)
      Gateway->>Pipeline: Process(Outgoing{from, to, text})
      Pipeline->>Pipeline: Billing reserve
      Pipeline->>Router: Route
      Router-->>Pipeline: Connector
      Pipeline->>Queue: Enqueue
      Queue-->>Gateway: OK
      Gateway-->>ExternalESME: submit_sm_resp (msgId)
    end
  else Credenciales inválidas
    Gateway-->>ExternalESME: bind_transceiver_resp (ESME_RINVPASWD)
    Gateway->>Gateway: Close connection
  end
end

```

6. Routing con Grupos y Fallback

```
flowchart TD
    A[Mensaje entrante] --> B{Evaluar reglas<br/>por prioridad}
    B -->|Regla 1 match| C{Destino?}
    B -->|Ninguna match| Z[REJECTD<br/>sin ruta]

    C -->|connector_id directo| D[Conector específico]
    C -->|group_id| E[Grupo de conectores]

    E --> F[Ordenar por peso<br/>weight]
    F --> G[Candidato 1<br/>mayor peso]
    F --> H[Candidato 2]
    F --> I[Candidato N]

    G --> J[Enviar al SMSC]
    J -->|éxito| K[ACCEPTED → DELIVERED]
    J -->|fallo transporte| L{¿Quedan<br/>candidatos?}
    L -->|sí| M[Reencolar<br/>try_count++]
    M --> H
    L -->|no| N[UNDELIV<br/>agotados candidatos]

    D --> J
```

7. Billing — Flujo de Crédito

```
flowchart LR
    A[Tenant] -->|balance| B{¿Modo?}
    B -->|prepaid| C[Reserve<br/>al encolar]
    C --> D[Mensaje en cola]
    D -->|aceptado| E[Debit<br/>débito real]
    D -->|rechazado| F[Credit<br/>devolver reserva]
    E --> G[Transacción<br/>audit log]
    F --> G

    B -->|postpaid| H[Sin reserva<br/>débito al aceptar]
    H --> I[Transacción<br/>audit log]
```

8. Flujo de un SMS end-to-end (resumen visual)

```
flowchart LR
    subgraph Entrada
        A1[HTTP POST] --> P
        A2[ESME bind] --> P
    end

    subgraph Pipeline
        P[Validar] --> R[Rutear]
        R --> B[Billing<br/>reserva]
        B --> Q[Encolar]
    end

    subgraph Cola
        Q --> W1[Worker 1]
        Q --> W2[Worker 2]
    end
```

```

    Q --> W3[Worker N]
end

subgraph Salida
    W1 --> S1[Sesión SMPP 1]
    W2 --> S2[Sesión SMPP 2]
    S1 --> SMSC1[SMSC A]
    S2 --> SMSC2[SMSC B]
end

subgraph DLR
    SMSC1 --> D[deliver_sm]
    SMSC2 --> D
    D --> DC[Correlación]
    DC --> WEB[Webhook]
    DC --> ESME[ESME entrante]
end

```

9. Estados del Mensaje (máquina de estados)

```

stateDiagram-v2
    [*] --> buffered : Pipeline encola
    buffered --> accepted : Worker acepta envío
    accepted --> delivered : DLR DELIVRD recibido
    accepted --> expired : DLR EXPIRED / Reconciler
    accepted --> undeliv : Sin candidatos (fallback agotado)
    accepted --> rejected : SMSC rechaza

    buffered --> failed : Error pipeline/cola

    delivered --> [*]
    expired --> [*]
    undeliv --> [*]
    rejected --> [*]
    failed --> [*]

```

10. Arquitectura de componentes

```

flowchart TB
    subgraph VM["VM (systemd)"]
        subgraph nginx["nginx :443"]
            FE["Frontend React<br>/var/www/smppgw"]
            API_PROXY["proxy_pass<br>/api → :8080"]
        end
    end

    subgraph smppgw["smppgw :8080 / :2775"]
        SRV["Server Role<br>API + ESME"]
        CONN["Connector Role<br>Worker + Session"]
    end

    subgraph data["Datos"]
        PG["PostgreSQL<br>:5432"]
        REDIS["Redis<br>:6379"]
    end
end

```

```
EXT[Clientes HTTP] --> nginx
EXT2[SMSC] --> smppgw
SRV --> PG
SRV --> REDIS
CONN --> PG
CONN --> REDIS
FE --> API_PROXY
```

11. Deploy — Secuencia de instalación

flowchart TD

```
A[Instalar dependencias<br/>PG, Redis, nginx] --> B[Crear BD y usuario]
B --> C[Compilar binario<br/>CGO_ENABLED=0]
C --> D[Transferir binario<br/>a /opt/smppgw/]
D --> E[Aplicar migraciones<br/>migrate up 0001-0007]
E --> F[Configurar systemd<br/>smppgw.service]
F --> G[Configurar nginx<br/>SSL + proxy]
G --> H[Compilar frontend<br/>npm run build]
H --> I[Transferir dist/<br/>a /var/www/smppgw/]
I --> J[Iniciar servicios<br/>systemctl start]
J --> K[Verificar<br/>curl + navegador]
```